

SOUTENANCE DE PROJET DE FIN D'ÉTUDES

Pour l'obtention du diplôme de Master Professionnel
en « Big Data & Cloud Computing » — BDCC

AI-Assisted Generation of Selenium Page Objects

A Structured Pipeline Compared Against Direct Large Language Model Generation

Soutenu le

[defence date]

par : [YOUR NAME]

Encadré par

[supervisor name(s)]

Membres du jury

[jury member 1]

[jury member 2]

[jury member 3]

PLAN

INTRODUCTION

- 1 GENERAL CONTEXT AND PROBLEM STATEMENT
- 2 METHODOLOGY, PLANNING AND EVALUATION PROTOCOL
- 3 FUNCTIONAL DESIGN
- 4 TECHNICAL DESIGN
- 5 IMPLEMENTATION
- 6 EXPERIMENTAL EVALUATION

PERSONAL ASSESSMENT • PERSPECTIVES • CONCLUSION

INTRODUCTION

A language model can write a page object.
~~The question is whether it should write all of it.~~

What is not in dispute

- A modern model reads HTML and produces plausible Java.
- I built it. It works. You will see its output.
- The idea is being pursued commercially.

What this project examines

- The task looks like one job. It is two.
- Structure: which elements exist, how they repeat, how to address them.
- Semantics: what a thing should be called.
- The two have different economics.

This project divides the task — and measures the division.



GENERAL CONTEXT AND PROBLEM STATEMENT

Problem statement

1

Locators must be authored.

Thirty interactive elements is thirty decisions, each needing a look at the markup. Mechanical, and unavoidable.

2

Locators are fragile.

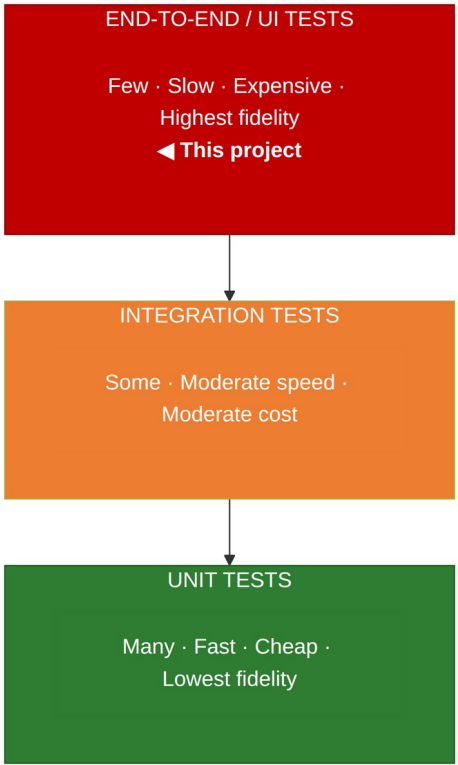
A locator is a claim about a document’s structure. One wrapper element invalidates a positional path; a copy edit invalidates a text match.

3

Locator quality is not uniform.

A dedicated test attribute has no reason to change. Visible text has every reason to. That gradient can be encoded as a rule.

The user interface is the most valuable layer to test and the most expensive. Both follow from one operation: locating an element.



The Page Object Model relocates the cost — it does not remove it

What it fixes

- No test contains a locator.
- All structural knowledge in one class per page.
- A changed identifier is a one-line change.

What it leaves behind

- Every page still needs a class.
- Every element still needs a locator and a method.
- The work is regular: find, choose, emit, repeat.

*Work that regular is a natural candidate for automation.
That observation is the origin of this project.*

The obvious solution — and four reasons to examine it rather than adopt it

Non-determinism

The same prompt twice may give different code. An artefact that varies between invocations cannot be meaningfully diffed or reviewed.

Hallucination

Models reference APIs that do not exist, because such code is statistically plausible. It looks right; only compilation reveals otherwise.

Cost scales with page size

The whole document is submitted every time. A recurring bill proportional to the application, not to the number of hard decisions.

No traceability

When a rule picks a locator, the reason is recoverable. When a model picks one, no account is available.

None of these says the model should not be used. They say where and how it is used has consequences — and that the question is answerable by measurement.

Objectives and research questions

RQ1

Can a deterministic pipeline, working from a live page's rendered HTML, produce page objects of a quality comparable to a language model given the same page?

RQ2

Where the deterministic layer cannot find a stable locator, can it detect that condition itself — and does escalating only those cases beat sending the whole page?

RQ3

On which measurable dimensions does each approach dominate, and what does that imply for how such systems should be built?

RQ2 is the contribution. Note that it has two halves: can the layer detect its own weakness, and is acting on that detection worth it?

Functional requirements

Test automation framework

- Drive a real browser against a live application.
- Two browsers, selectable without touching test code.
- Headless by default — local and CI identical.
- Page Object Model; no locator in any test.
- Reusable multi-page journeys (flows).
- All URLs, credentials and messages externalised.
- HTML report, screenshot on failure.
- API tests alongside UI tests.
- Runs on every push through CI.

Generation pipeline

- Retrieve fully rendered HTML, including behind a login.
- Extract interactive elements; exclude hidden fields.
- Decompose into header, footer and body.
- Detect repeating components; model each once.
- Select the most durable locator by fixed priority.
- Record which rule produced each locator.
- Mark as low-confidence anything matched only by text.
- Emit compilable Java extending the same base class.
- Escalate only low-confidence elements to a model.
- Two model backends behind one interface.
- Degrade gracefully on model failure.
- Provide the naive baseline mode for comparison.

Non-functional requirements

Determinism

Identical input must produce identical output.

Traceability

Every locator attributable to an explicit rule.

Cost proportionality

Model cost scales with ambiguity, not page size.

Isolation

Generated artefacts separated from reviewed code.

Build safety

Unvalidated model output can never break the build.

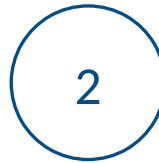
Portability

Runs unmodified locally, on cloud CI and a self-hosted agent.

Secret hygiene

Credentials supplied by environment, never committed.

Build safety was added mid-project, after it was recognised that writing unvalidated model output into the compiled source tree would let a single non-compiling response break the whole build, including CI.



METHODOLOGY, PLANNING AND EVALUATION PROTOCOL

Development methodology

Implement the smallest useful capability → exercise it against a real, live website → observe the failures → diagnose each to its root cause → correct with a scoped change.

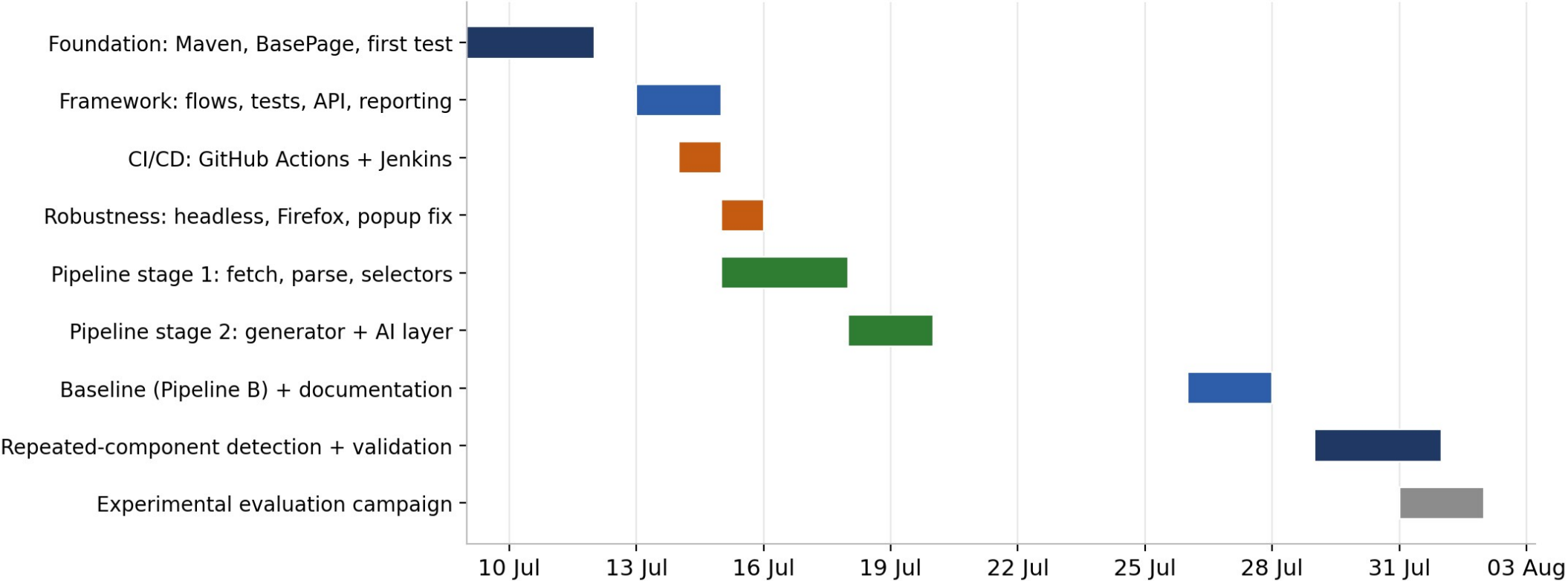
The insistence on real sites is methodologically significant

- Every substantive defect in Chapter V was found this way. None would have surfaced on a clean, tutorial-quality test page.
- A product named `test.allTheThings()` T-Shirt (Red) produced attribute values that are not legal in Java identifiers.
- The second target site used `data-qa` where the first used `data-test` — revealing that the convention had been hard-coded rather than configured.

Two target applications, chosen deliberately: SauceDemo is purpose-built and cleanly instrumented — a favourable case. `automationexercise.com` is messier and more realistic. Evaluating both distinguishes behaviour on an idealised page from behaviour on a real one.

Planning

Project timeline, reconstructed from version-control history



A comparison whose protocol is not stated is not a result

Nine metrics, fixed before measuring

- Recall · Precision · Locator quality
- Compilability · Executability · Duplication
- Cost · Latency · Determinism

Fairness of the baseline

- A genuinely helpful prompt — prefer stable attributes, skip hidden inputs, extend the same base class.
- Formatting artefacts stripped before evaluation, not penalised.
- Both pipelines receive identical HTML, same session, same day.

The conflict of interest

Pipeline A is my own system. I designed the metrics and I took the measurements.

Three commitments

- The metric set was fixed before the campaign began.
- Defects in my pipeline are logged with the same severity scale as defects in the baseline.
- Every artefact is retained, so any measurement can be independently recomputed.

Threats to validity

Construct

The metrics are proxies. Compilability and executability are strong evidence of usability but do not establish maintainability — the property that ultimately matters, and which could not be measured in the time available.

Internal

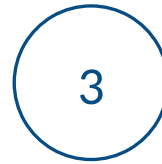
Ground-truth counts were established by a single observer. A second independent counter would strengthen them. A hosted model may also change without notice, independently of the experiment.

External

Two sites, one page examined in depth. Both are English-language e-commerce sites of moderate complexity. Single-page applications with heavy dynamic re-rendering, or shadow DOM, are not covered.

Reliability

Pipeline A’s output is byte-reproducible, so its measurements reproduce exactly. Pipeline B’s cannot be, by the very property under study — so its results are reported per run, and the variance is itself treated as a result.



FUNCTIONAL DESIGN

Actors

QA Engineer

The primary human actor. Executes the suite, initiates generation, and — crucially — reviews low-confidence output. The system is designed to be human-in-the-loop, not autonomous.

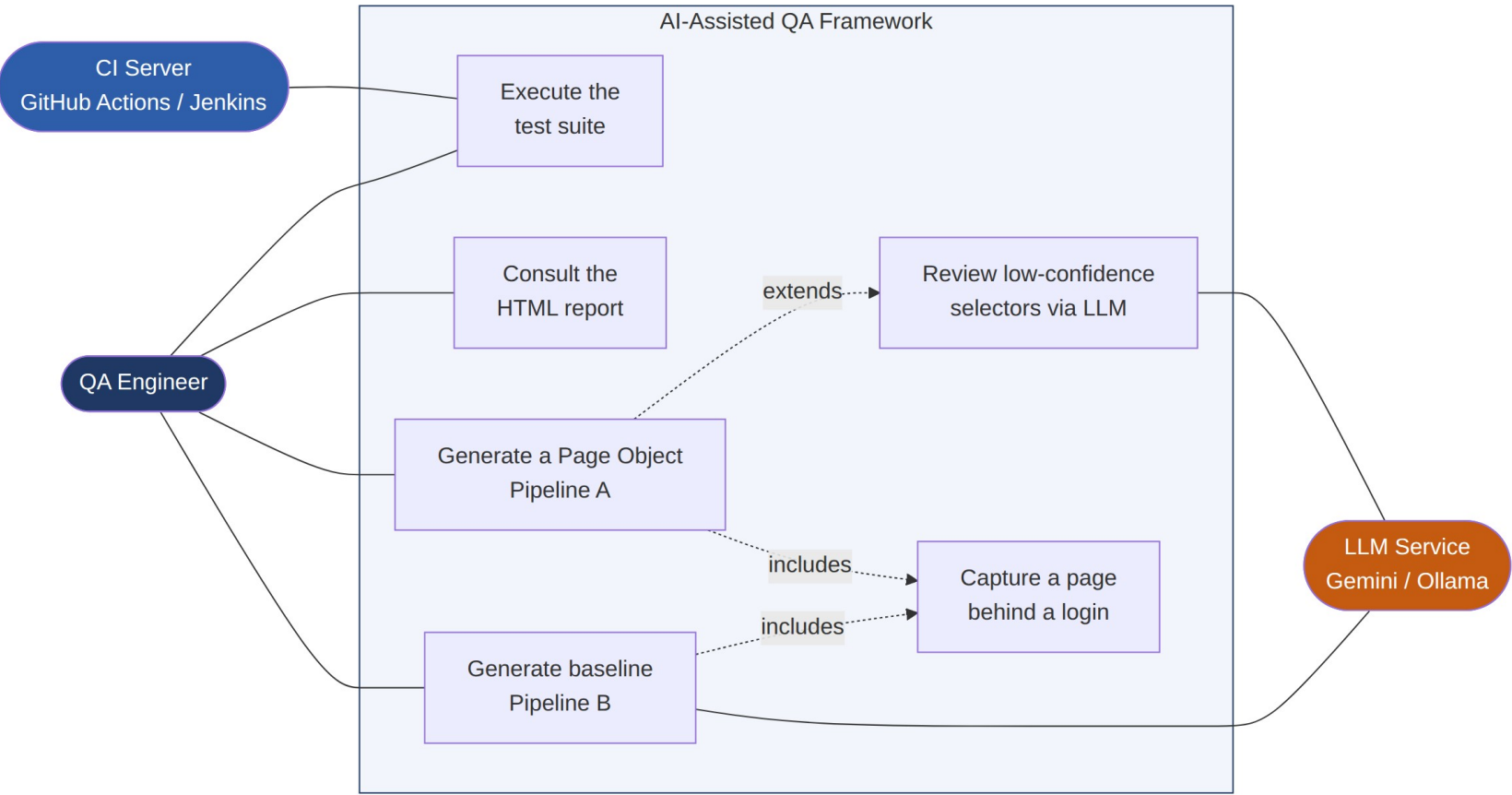
CI Server

A secondary, non-human actor. Executes the suite automatically on every change. Realised by two independent implementations: GitHub Actions and Jenkins.

LLM Service

A secondary, external actor providing judgement on request. Gemini in the cloud or a local Llama via Ollama. Its availability is not guaranteed and the system is designed accordingly.

Use case diagram



The contribution: confidence-gated escalation

Four strategies, tried in order

data-test / data-qa

exists only for testing — no reason to change

id

intended to be unique, reasonably stable

name

carries form semantics, fairly stable

visible text

changes with a copy edit; changes entirely under translation

The gate

isLowConfidence()

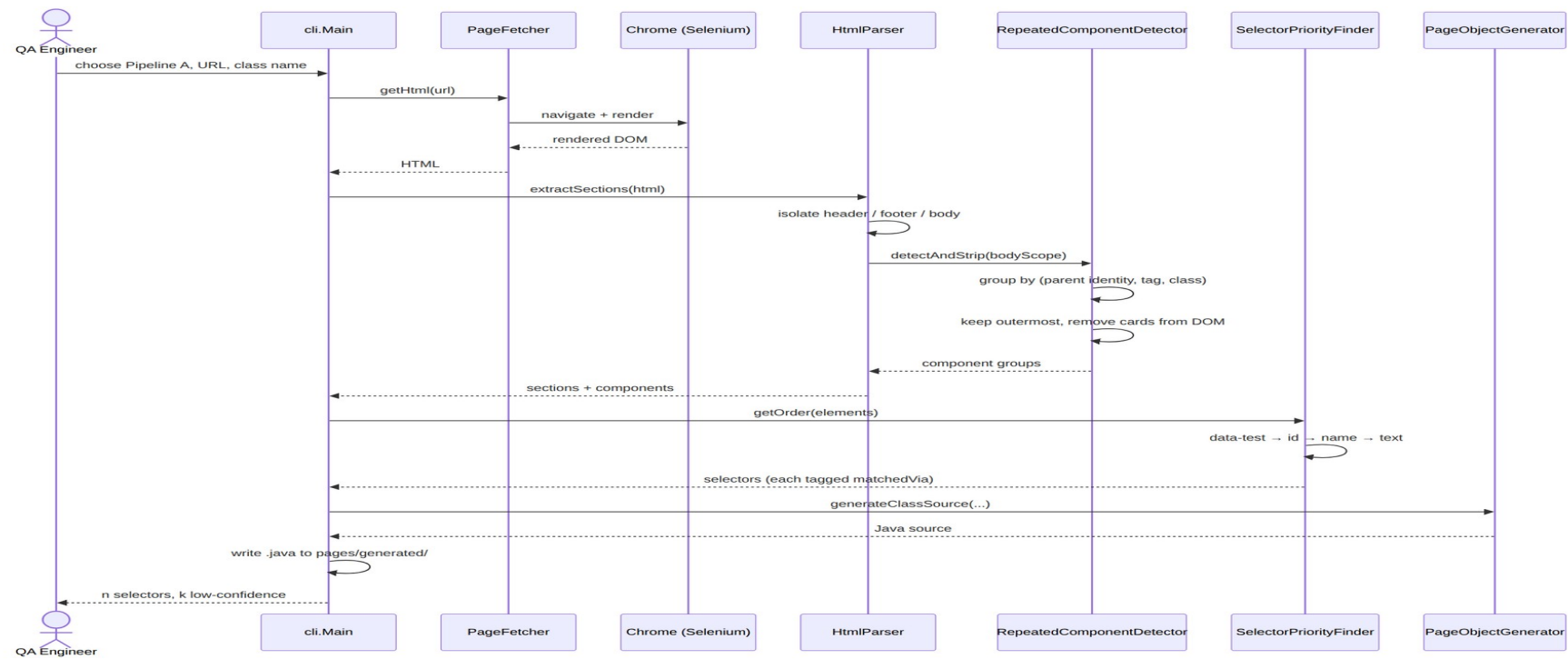
Only elements that fell all the way to a text match are sent to the model.

The model's cost therefore scales with ambiguity, not with page size.

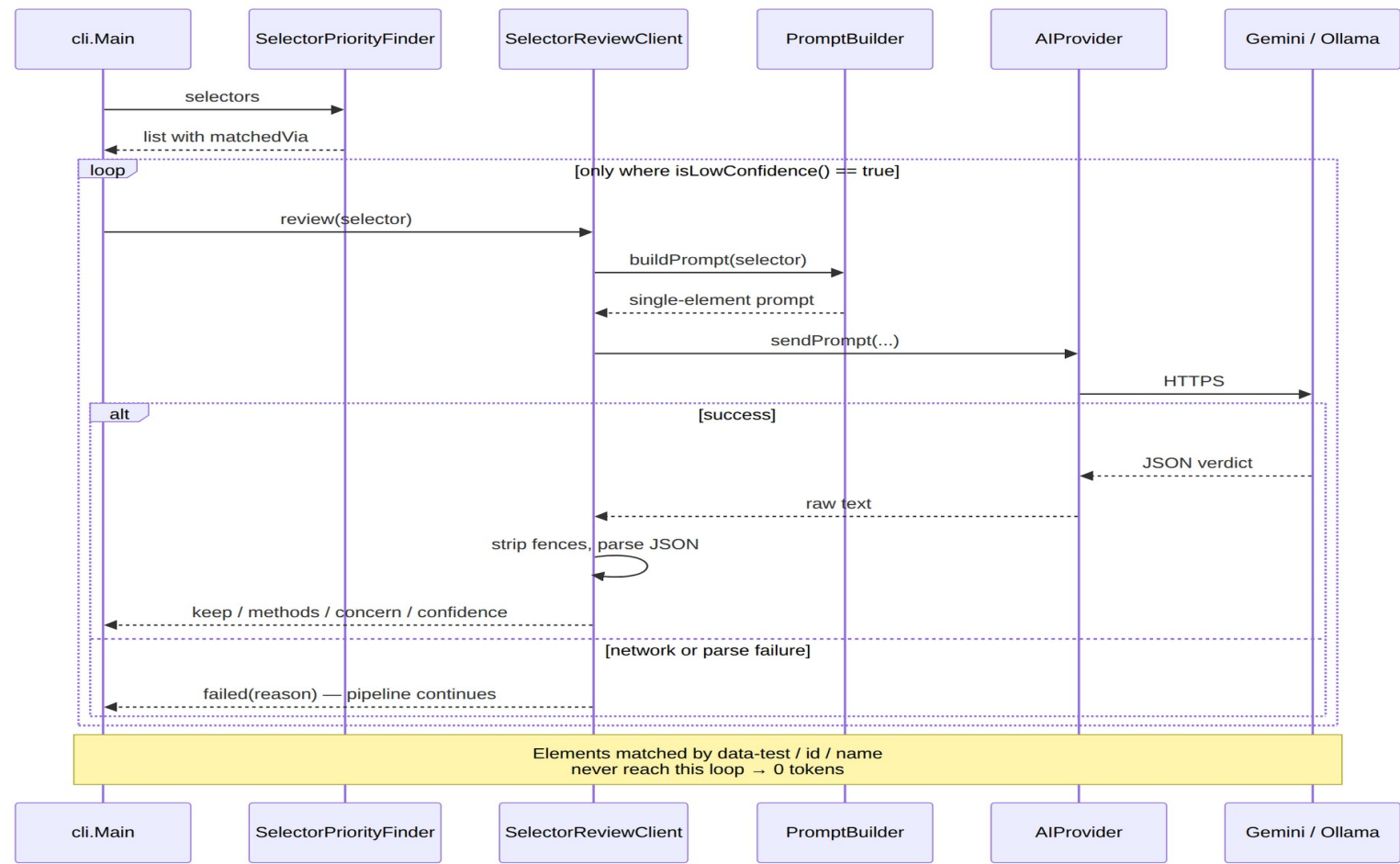
One field — `matchedVia` — records which rule produced each locator. That single field is what makes the output traceable and what makes gating possible.

On a well-instrumented page the number of model calls is zero — and so is the cost.

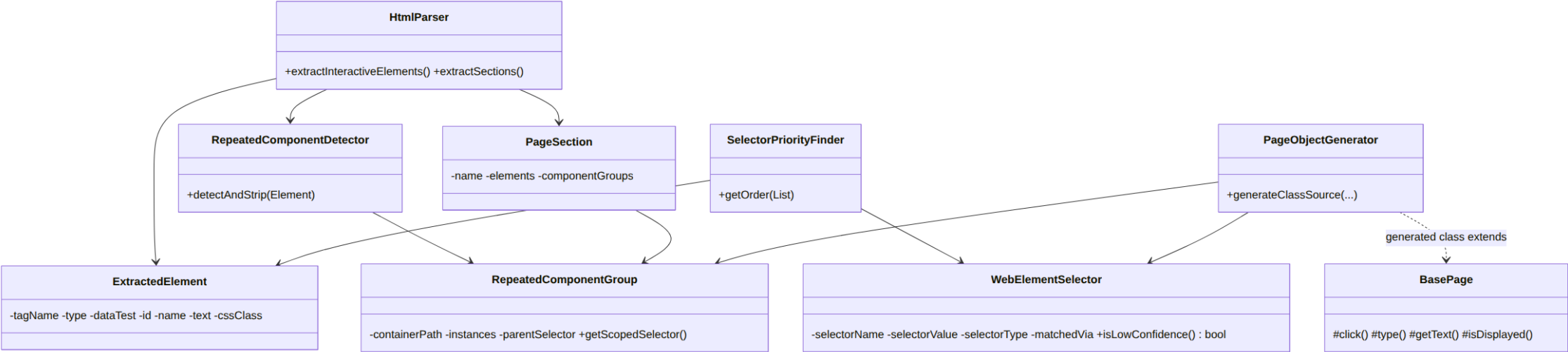
Sequence diagram — Pipeline A generation



Sequence diagram — confidence-gated review



Class diagram — parser and generator packages



Data carriers. `ExtractedElement` holds what was observed. `WebElementSelector` holds what was decided — including `matchedVia`, the field that makes gating possible.

Processors. Parser, detector, priority finder and generator: one responsibility each, none knowing the others' internals.

The AI abstraction. `AIProvider` is a single-method interface; the review client depends on it alone. Substituting a cloud model for a local one is a one-line change.

Detecting structure, not just elements

The naive approach

Group elements by CSS class. Fails on real pages: framework classes such as row, container and col-sm-4 recur across entirely unrelated regions.

What actually characterises a list

Members share a parent. The key is (parent identity, tag, first class), with a threshold of three: two similar siblings are a coincidence, three are a structure.

Two further rules

Outermost wins. An inner element may repeat inside an outer one that also repeats. The outer element is the true card and already contains everything the inner one holds.

Cards are removed as they are captured. Whatever is extracted afterwards therefore cannot contain a card's elements. No filtering pass is required, and the two extractions cannot disagree — an entire class of bug removed rather than defended against.

Result on the evaluated page: one menu component covering 4 items, one product component covering 6 — instead of thirty near-identical fields.



TECHNICAL DESIGN

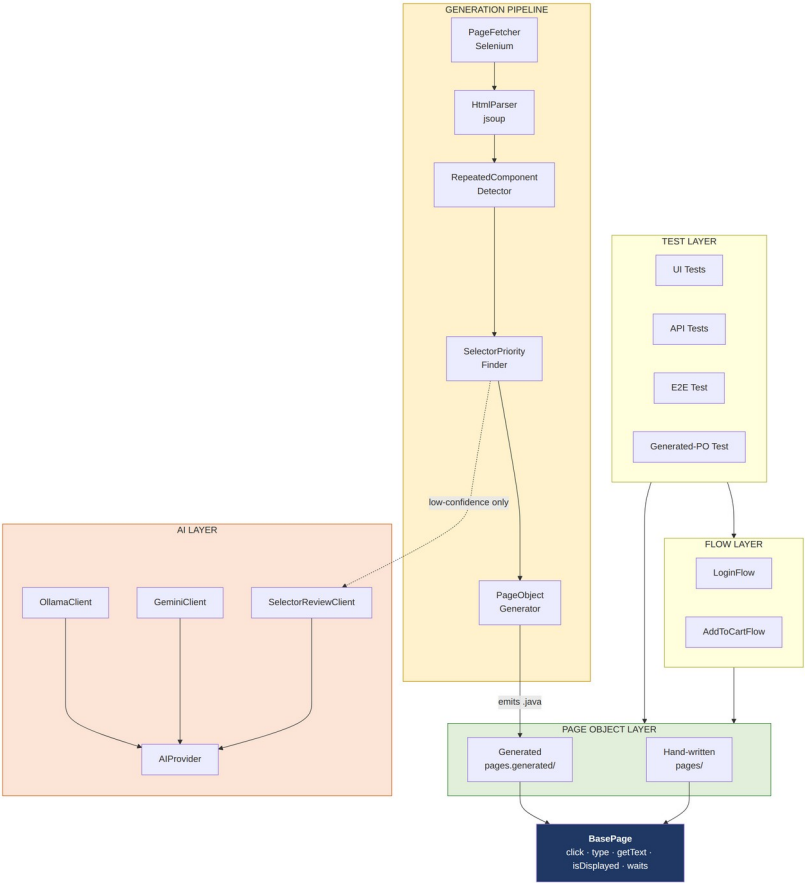
Layered architecture

Test framework

Base page, page objects, flows, tests — UI and API — with reporting and two CI pipelines. Not decoration: it is the harness that proves generated artefacts are usable in a real suite.

Generation pipeline

Fetch the rendered DOM, section the page, detect repeating components, select locators, emit Java. Four stages, one responsibility each.



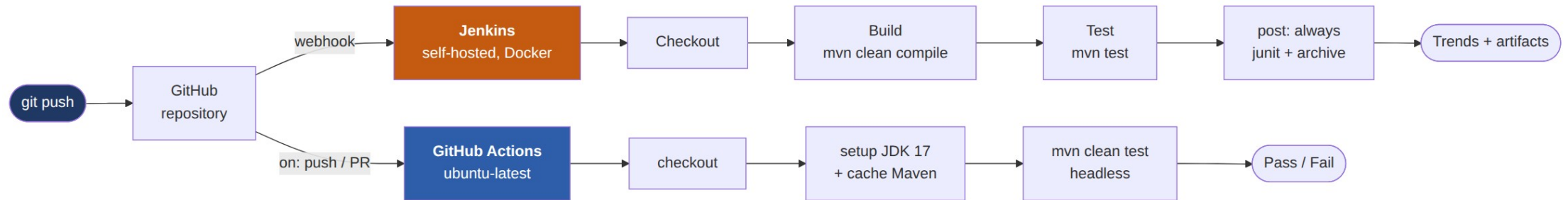
AI layer

One interface, two backends — Gemini in the cloud, Llama via Ollama locally. A model failure is returned as a result, never thrown, so the deterministic output always survives.

Technologies

Language / build	Java 17 · Maven	Reporting	ExtentReports 5.1
Browser automation	Selenium WebDriver 4.21	HTTP client	OkHttp 4.12
Test framework	TestNG 7.10	Cloud model	gemini-3.1-flash-lite
API testing	RestAssured 5.4	Local model	Llama 3.1 8B via Ollama
HTML parsing	jsoup 1.17	Continuous integration	GitHub Actions · Jenkins (self-hosted, Docker)
Test data	Datafaker 2.4	Version control	Git / GitHub

Continuous integration and delivery



GitHub Actions runs on every push to main: a clean Ubuntu machine, JDK 17, headless. The value of a clean machine is easily understated — it proves the project builds from the repository alone.

Jenkins runs self-hosted in Docker on a webhook, with Checkout / Build / Test as separate stages so the dashboard distinguishes a compilation failure from a test failure.

A fast offline gate runs the generator's own unit tests first — no browser, no network — so a red build immediately says which kind of failure it is.



IMPLEMENTATION

The generator in operation

```
$ mvn -q compile exec:java -Dexec.mainClass=cli.Main

1 - Pipeline A: structured extraction + generate Page Object
2 - Pipeline A + AI review of low-confidence selectors
3 - Pipeline B: naive raw-HTML AI baseline
Choose an option (1/2/3): 1

Detected 2 repeated component pattern(s):
  Pattern: a.bm-item          -> 4 instances
  Pattern: div.inventory_item -> 6 instances
Section Body: 3 selectors, 0 low-confidence
Written: src/main/java/pages/generated/...Body.java
```

The last line is the measurement. Three selectors, zero low-confidence — so nothing met the escalation condition, no prompt was ever constructed, and the model was never called. Cost for this page: zero tokens.

For pages behind a login, the tool opens a visible browser, waits for the engineer to authenticate and navigate by hand, and captures whatever page they reached — rather than encoding assumptions about any particular login form, which would work for exactly one site.

A generated page object

```
package pages.generated;

public class SauceDemoInventoryPageBody extends BasePage {

    private final By byReactBurgerMenuBtn = By.cssSelector("[id='react-burger-menu-btn']");
    private final By byShoppingCartLink    = By.cssSelector("[data-test='shopping-cart-link']");

    public void clickShoppingCartLink() { click(byShoppingCartLink); }

    // "div.inventory_item" repeated 6 times on the page this was generated from.
    public List<InventoryItemItem> getInventoryItemItemList() {
        return driver.findElements(By.cssSelector("div.inventory_list > div.inventory_item"))
            .stream().map(InventoryItemItem::new).collect(Collectors.toList());
    }
}
```

Extends the same BasePage as hand-written page objects, and lands in a dedicated pages.generated package so machine output is never mistaken for reviewed code.

Validated end-to-end by generated page objects alone

```
List<ProductsPageBody.ColSm4Item> products = body.getColSm4ItemList();
actions.moveToElement(firstProduct.getRoot()).perform(); // hover, then click
firstProduct.clickAddToCart();
body.clickContinueShopping();
...
int expectedTotal = Integer.parseInt(price) * Integer.parseInt(quantity);
Assert.assertEquals(Integer.parseInt(total), expectedTotal);
```

A real two-product purchase flow on automationexercise.com. Navigation and interaction use generated page objects only — no locator typed by hand. The assertion checks that price × quantity equals the displayed cart total. The cart-side assertions do use hand-written locators, because no cart page object was generated.

getRoot() is the escape hatch the generator provides for anything it could not anticipate — here, hovering a card to reveal a control that only appears on hover.

Test execution and reporting

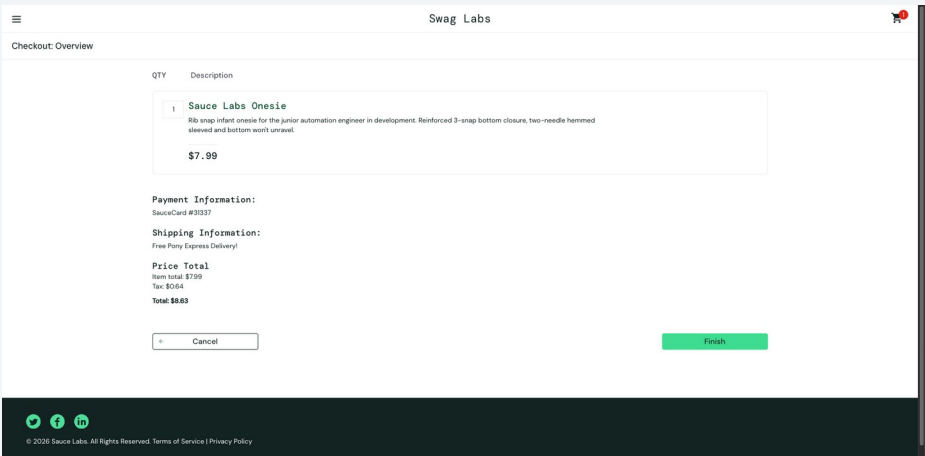
SauceDemo Automation Suite

ExtentReports · 15 July 2026, 16:45

✓ loginTest	0.82 s
✓ invalidLoginTest	0.63 s
✓ loginUsingValidCredentialsTest	0.70 s
✓ loginUsingInvalidCredentialsTest	0.61 s
✓ loginUsingLockedUserTest	0.67 s
✓ logoutCorrectlyTest	11.19 s
✓ addingItemsToCartTest	0.86 s
✓ fillingCustomerInfoTest	1.08 s
✓ getThankYouMessage	1.06 s
✓ PingTest	1.52 s
✓ checkout	1.92 s

11 tests · 11 passed · 0 failed

Screenshot captured on failure



The listener captures the browser state at the moment of failure and attaches it to the report. It obtains the driver by reflection over the test's base class — the framework's weakest point, acknowledged in the report rather than concealed, and recorded as future work.

[[Replace the left panel with your own screenshot of reports/ExtentReport.html — it renders fully styled when you open it locally.]]

Testing the generator itself

37

unit tests

0

browsers, networks or live sites required

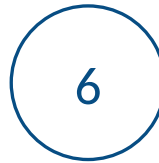
3

stages covered: extraction, locator selection, code emission

A project about testing whose own generator is untested is in an awkward position. For most of this project, this one was.

Written against behaviour that has actually been wrong

- that a data-qa element yields a data-qa selector;
- that a card which is itself the interactive element is addressed directly;
- that an apostrophe in page text does not produce a broken XPath literal;
- that text differing between instances of a component names neither the method nor the locator.



EXPERIMENTAL EVALUATION

Experimental setup

Date of runs	31 July 2026
Model	gemini-3.1-flash-lite
Target page	SauceDemo inventory page (authenticated)
Section scope, Pipeline A	Body region
Runs per pipeline	3, consecutive, no intervening change
Environment	IDK 17 • Chrome 150 • macOS

Reproducibility

The rendered DOM both pipelines consumed was captured during the campaign and is retained, together with every generated artefact and the console telemetry.

Scripts accompanying the report re-derive the element counts, the locator resolution, the compilability results and the digests from those files alone — no network, no model call.

Scope, stated plainly: one page examined in depth, three runs per pipeline. Enough for categorical findings. Not enough for quantitative generalisation — and none is claimed.

Result 1 — determinism: the same command, run three times

Pipeline A

420c015a... · 420c015a... · 420c015a...

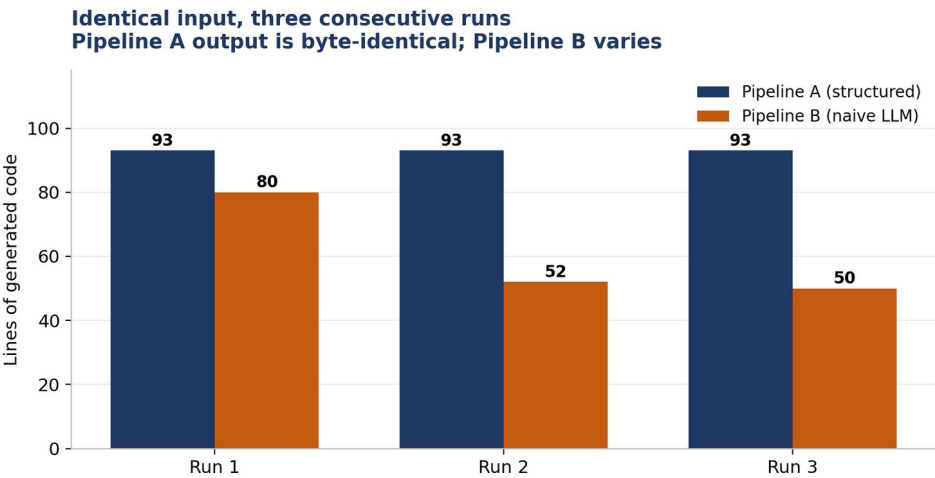
byte-identical

Pipeline B

19ea0a03... · dedd5d21... · 65662d40...

all three differ

The variation is not marginal: 80 / 52 / 50 lines of code, 11 / 17 / 17 declared fields — and run 2 declared a different package, so the artefact would land in a different directory on every regeneration.



Prompt tokens were identical on all three runs (13,115). The input did not vary. The output did.

Result 2 — a Selenium method that does not exist

```
private final By inventorySidebarLink = By.dataTest("inventory-sidebar-link");
```

org.openqa.selenium.By declares exactly eight static factory methods. dataTest is not one of them.

11 ×

occurrences, run 1

15 ×

occurrences, run 2

3 / 3

Pipeline A compiles

1 / 3

Pipeline B compiles

Pipeline A cannot produce this defect. It writes `By.cssSelector` and `By.xpath` from a fixed template, over selectors observed in the document. There is no mechanism by which a non-existent method could enter its output. That is the difference between emitting code and predicting it.

Result 3 — cost scales with ambiguity, not with page size

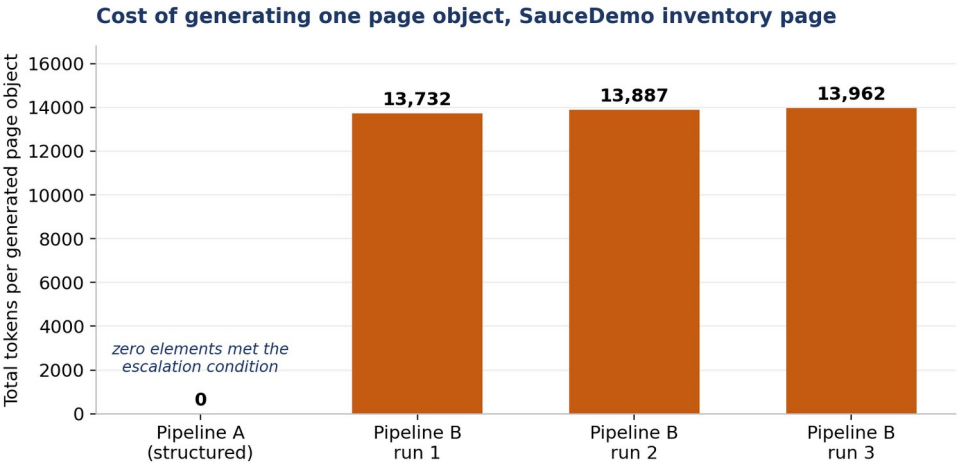
0

tokens — Pipeline A

13,732

tokens — Pipeline B, one run

Pipeline A consumed nothing because the page yielded zero low-confidence selectors: every element was addressable by a test attribute or an id, so nothing met the escalation condition. This is the gate operating exactly as designed.



In currency. At the published price for this model, the three baseline runs cost \$0.0042, \$0.0044 and \$0.0046 — \$0.0132 together, against \$0.00. Trivial for one page. But the baseline’s prompt cost is a function of document size and is paid on every regeneration, while Pipeline A’s would rise only if the page became less well instrumented. Two hundred page objects regenerated weekly is on the order of 140 million input tokens a year, against zero.

Result 4 — comparable coverage, different structure

Pipeline A

3 page-level selectors, plus two component classes: a menu covering 4 items and a product card covering 6, each scoped to one card's root element.

Indifferent to how many products exist.

Pipeline B

6 individually hard-coded methods — `addBackpackToCart()`, `addBikeLightToCart()` — one per product present at the moment of generation.

The catalogue is encoded into the interface.

Add a seventh product and the baseline's class is silently incomplete.
Remove one and it has a method that can no longer work.

The functional coverage is comparable. The structural quality is not.

Defects in my own pipeline

Reporting only the baseline's defects would make this an advertisement rather than an evaluation.

```
public void clickAllItems() {  
    WebElement el = root.findElement(By.cssSelector("a.bm-item"));    // root IS the a.bm-item
```

jsoup's select() includes the element itself; Selenium's findElement() searches descendants only. The generated locator searches inside an anchor for another anchor.

0 / 4

instances resolved — every call throws

1 / 6

a second defect: a component role named from
one card's product text

19 / 19

and the one baseline run that compiled had every locator correct

It was not caught during the campaign because no test ever executed that component. An artefact that compiles, reads correctly and is never run will not reveal a defect of this kind.

What the measurements support

Structure

Which elements exist • how they repeat • how to address them durably

Better solved deterministically. Determinism buys reproducibility, traceability and zero marginal cost.

`click6Polo()` `getText5HM()`

Semantics

What a thing should be called • what an unfamiliar convention means

Where a language model contributes something rules cannot supply. It named things well, and inferred a convention unprompted.

`addBackpackToCart()`

The two approaches fail in different places. Neither result is a verdict on language models — it is a boundary.

RQ1: yes, with one qualification. RQ2: decisively, on this page. RQ3: each dominates on different dimensions.

Personal and professional assessment

Diagnostic reasoning

Every defect in Chapter V was traced to a root cause rather than patched at the symptom — including one where the cause turned out to be outside the DOM entirely.

Working against reality

Real sites, not fixtures. Every substantive defect was found that way, and none would have appeared on a tutorial page.

Measuring rather than asserting

Fixing a metric set before measuring, and reporting the results that went against my own system.

New technologies

Selenium, TestNG, jsoup, two CI platforms, and two model backends behind one interface.

[[Rewrite these four in your own words before the defence — the jury listens closely to this slide, and it must sound like you.]]

Perspectives

Deterministic layer builds the skeleton → the model names and prunes it

The architecture neither pipeline implements — and the principal recommendation of this work

Why this is the right shape

- The model receives a compact structured description, not raw markup — a fraction of the token cost.
- Its output is names, not code — so a hallucinated method cannot enter the artefact at all.
- The pipeline already knows exactly which names it could not derive.

Also next

- Extend the campaign to the full six-page set, enabling quantitative rather than categorical claims.
- Golden-file tests over whole saved pages.
- Support for <select> through Selenium's Select class — the one dimension where the baseline had broader coverage.
- Self-healing locators at runtime.

Conclusion

The boundary between the two approaches is the boundary between structure and semantics.

A system that respects that boundary is better than either pure approach. Identifying where it falls — with measurements rather than assertion — was the objective of this work.

Pipeline A wins on

Determinism • cost • reliability • traceability • structural abstraction

Pipeline B wins on

Method naming • inferring an unfamiliar convention unprompted

Both produced defects. Both are reported.



Merci pour votre attention

Questions

All artefacts, the measurement scripts and the evaluation workbook accompany the report.
Every figure in Chapter VI can be recomputed offline from the retained files.

BACKUP — DETAIL SLIDES FOR QUESTIONS

All measurements, page P1

Metric	Pipeline A	Pipeline B
Determinism (3 runs)	byte-identical	all three differ
Compiles unmodified	3 / 3	1 / 3
Prompt tokens	0	13,115 per run
Output tokens	0	617 / 772 / 847
Latency	no model call	3,125 / 3,342 / 2,918 ms
Cost, 3 runs	\$0.00	\$0.0132
Elements addressed (body scope)	25 / 25	—
Locators resolving	5 / 7 → 7 / 7 after fixes	run 3: 19 / 19
API calls	0	1 per run

Ground truth: 29 interactive elements page-wide; 26 in the body region, of which 25 are of the kinds the parser collects.

Defect log — both pipelines, same severity scale

ID	Pipeline	Defect
D1	B	Hallucinated By.dataTest — 11× run 1, 15× run 2. Neither compiles.
D2	B	Non-deterministic output from identical input; three different digests.
D3	B	type() called on a <select>; all three runs. A systematic error, not sampling noise.
D4	B	Fabricated a locator and documented the guess in a comment.
D5	B	A 503 from the model service destroyed a complete run, including the manual login.
D6	A	Component role named from one card's product text: 1 of 6 instances resolve.
D8	A	Card that IS the interactive element searched inside itself: 0 of 4 resolve.
D9	A	data-qa element emitted a [data-test=...] locator. Latent — never fired on either site.
D11	A	No escaping of quotes in emitted XPath / Java literals. Latent.
D12	B	When it compiled, all 19 of its locators were correct.

D8–D12 were found in a systematic post-campaign audit. All four Pipeline A defects have since been fixed, each with a regression test.

Repeated-component detection — activity diagram

